UPO v3.3.0 – Nachtrag M15 bis M17 für GrokBuild

Aktionsebene mit überprüfbarer Aufgabe, Messungen am ausgeführten Kern und eine Beobachtung zur Sensorik

Für: Rick (DarkStarMass) • Stand: 17. Oktober 2026 • Grundlage: grok-workspace.zip (C1 bis C19), upo-lauf.json, upo-stress-50.csv, upo-agenten.csv

1. Ausgangslage

Der Stand mit C15 bis C19 ist nachgemessen und stimmt: alle Selbsttests grün, die App-CSV deckt sich in 50 von 50 Zeilen mit meinem Node-Lauf, 27 ältere Läufe sind bitgleich, die Härtung des JSON-Imports hält in allen 17 Angriffs- und Fehlerfällen. Offen ist die Aktionsebene: Im Stresstest steht der Decoder in **50 von 50** Iterationen auf UNDECIDED, auch mit Ruhe-Referenz. Das ist bei zufälligen Aktionsvektoren und einer kalibrierten Schwelle erwartbar und ehrlich, aber die Ebene entscheidet damit nie. Dieser Nachtrag gibt ihr eine überprüfbare Aufgabe und zeigt dabei eine Grenze der Sensorik, die ich beim Messen gefunden habe.

2. Messungen zur Machbarkeit

Ich habe aus Referenzszenarien Prototypen berechnet (Szenario durch den Adapter, Triage-Fusion, normiert, wie in `groundTruthFromScenario`) und die Endzustände echter Läufe gegen diese Prototypen decodiert (Seeds 42, 101, 2026, 7777, 99999; Schwelle 0.0916 aus C16).

Versuch	Treffer	Marge ab Schwelle	Marge min / median
9 Szenarien (3 Umgebungen mal 3 Rauschstufen), Zufall 11 %	16 von 36	0 von 36	0.000 / 0.004
3 Szenarien (nur Umgebung 0.2 / 0.8 / 1.4), Zufall 33 %	90 von 90	90 von 90	0.128 / 0.165

Grund für den Unterschied: Der Kosinus des Konsens-Vektors zwischen Rauschstufe 0.1 und 3.0 liegt bei gleicher Umgebung bei 0.981 bis 0.990. Die Prototypen mit gleicher Umgebung sind praktisch identisch (Kosinus 0.98 bis 1.00); zwischen den Umgebungsstufen liegen sie bei 0.32 bis 0.82. Der Zustandsraum bildet also im Wesentlichen nur den Umgebungsstress ab, der Rauschpegel ist fast unsichtbar, obwohl der Stresstest ihn von 0.10 bis 4.02 variiert.

> **Folge**
>
> Mit der heutigen Sensorik sind drei Aktionen sauber begründbar, neun nicht. Das ist kein Fehler im Kern, sondern eine Grenze der Merkmale. M15 nutzt, was tatsächlich unterscheidbar ist; M16 macht das Rauschen beobachtbar.

3. Übersicht

ID	Maßnahme	Dateien	Prio	Aufwand
M15	Aktionsvektoren aus Referenzszenarien (gebunden / ungebunden)	scenario.ts, adapter.ts, config.ts	1	mittel
M16	Rauschen beobachtbar machen (Qualitätskanal)	sensors.ts, adapter.ts, triage.ts	2	mittel
M17	Export-Konsistenz	studies.ts, UI	3	klein

Reihenfolge: M15 zuerst (nutzt nur vorhandene Messwerte), danach M16, danach M17. Neue Korrekturen als C20 bis C22 in `assumptions.ts`. Welche Aktion zu welchem Szenario gehört, ist eine Designentscheidung von dir; die Belegung unten ist ein Platzhalter-Vorschlag.

M15 Aktionsvektoren aus Referenzszenarien

Dateien: `scenario.ts, adapter.ts, config.ts, selfcheck.ts` **Priorität:** 1

Befund (gemessen)

Die neun Aktionsvektoren sind zufällige Einheitsvektoren (Marge unter Zufallszuständen: Median 0.019, 99. Perzentil 0.092). Der Decoder entscheidet deshalb im Stresstest nie (50 von 50 UNDECIDED). Mit drei Prototypen aus den Umgebungsstufen 0.2, 0.8 und 1.4 trifft er in 90 von 90 Läufen (5 Seeds, 3 Rauschstufen, 2 interne Stressstufen), die Marge liegt zwischen 0.128 und 0.223, also über der Schwelle.

Warum das geändert werden soll

Eine Aktion bekommt erst dann eine Bedeutung, wenn sie an eine nachprüfbare Eigenschaft der Eingabe gebunden ist. Drei Aktionen lassen sich mit der heutigen Sensorik belegen; die übrigen sollten ehrlich als ungebunden gelten und nicht mit Zufallsvektoren so tun, als würden sie entscheiden. Wichtig: Die Schwelle muss über genau die gebundene Menge kalibriert werden, weil sich die Nullverteilung der Marge mit der Zahl der Kandidaten ändert.

Änderung (Vorschlag, an eure Struktur anpassen)

```
// scenario.ts
export type ActionBinding = { action: ActionName; scenario: Scenario };
export const ACTION_BINDINGS: ActionBinding[] = [      // Platzhalter-Belegung
  { action: "MOTOR_PROCEED_GENTLY",
    scenario: { env: 0.2, noise: 1.0, internal: 0.3, seed: 0 } },
  { action: "PROMPT_LLM_CAUTION",
    scenario: { env: 0.8, noise: 1.0, internal: 0.3, seed: 0 } },
  { action: "MOTOR_STOP_IMMEDIATELY",
    scenario: { env: 1.4, noise: 1.0, internal: 0.3, seed: 0 } },
];
// adapter.ts: Prototyp je gebundener Aktion = groundTruthFromScenario(this, scenario)
// (am gleichen Adapter, daher pro Seed neu). Uebrige Namen: Status UNBOUND, nehmen am
// Decode nicht teil, erscheinen in der UI als "ungebunden".
decodeDetailed(v) { /* best, runnerUp nur ueber gebundene Prototypen;
                      margin >= minMargin sonst UNDECIDED */ }
// Schwelle: calibrateNullMargin() ueber genau die gebundene Menge
// (Quantil 0.99) neu bestimmen
```

Abnahmekriterium

Test T12: 3 Szenarien mal 3 Rauschstufen mal 5 Seeds mal 2 interne Stressstufen ergeben 90 von 90 Treffern mit Marge über der Schwelle (gemessen 90 von 90, Marge min 0.128). Test T13: Unter Zufallszuständen entscheidet der Decoder höchstens in 2 Prozent der Fälle (mit der neu kalibrierten Schwelle). Test T14: Ungebundene Aktionen erscheinen nie als Ergebnis. Stresstest: Die dekodierte Aktion wechselt mit der Umgebung (mindestens 2 verschiedene Aktionen über die 50 Iterationen). Gewinner und Konfidenzen bleiben bitgleich zum Stand C19.

M16 Rauschen beobachtbar machen

Dateien: `sensors.ts, adapter.ts, triage.ts` **Priorität:** 2

Befund (gemessen)

Rauschpegel 0.1 gegen 3.0 verändert den Konsens-Vektor nur um 1 bis 2 Prozent (Kosinus 0.981 / 0.990 / 0.990 für Umgebung 0.2 / 0.8 / 1.4). Das 9-Szenarien-Raster kollabiert auf 3 Prototypen (16 von 36 Treffer, Marge median 0.004, 0 von 36 über der Schwelle).

Warum das geändert werden soll

Wenn der Zustand den Rauschpegel nicht abbildet, kann keine nachgelagerte Ebene darauf reagieren. Ein Qualitätsmerkmal, das aus den Rohdaten selbst gewonnen wird (nicht aus dem Parameter `noise`), macht

die Signalqualität sichtbar. **Dieser Vorschlag ist noch nicht gemessen**; die Abnahme prüft ihn deshalb ausdrücklich.

Änderung (Vorschlag, an eure Struktur anpassen)

```
// sensors.ts: Qualitaetsmerkmal aus den Rohdaten (Streuung der zweiten Differenzen)
function residualStd(x: number[]): number {
  const d = x.slice(2).map((v, i) => v - 2 * x[i + 1] + x[i]);
  return Math.sqrt(d.reduce((s, v) => s + v * v, 0) / d.length);
}
const quality = [residualStd(visual), residualStd(auditory), residualStd(imuDeviation)];
// adapter.ts: vierte Modalitaet "quality" (3 Merkmale -> 512D, eigene Gewichte), Prior 0.6
// triage.ts: Energie wie bei den anderen Kanaelen (log1p),
// Prior in MODALITY_PRIORS ergaenzen
```

Abnahmekriterium

Kosinus des Konsens-Vektors zwischen Rauschstufe 0.1 und 3.0 liegt unter 0.9 (vorher 0.98 bis 0.99). 9-Szenarien-Raster: mindestens 30 von 36 Treffer mit Marge über der Schwelle. Falls die Messung scheitert, ist der Qualitätskanal im Dokument als Fehlversuch mit Zahlen zu vermerken statt still zu entfernen. Gewinner und Konfidenzen bleiben bei ausgeschaltetem Kanal bitgleich zum Stand C19.

M17 Export-Konsistenz

Dateien: `studies.ts, UI` **Priorität:** 3

Befund (gemessen)

Die Spalte heißt in der CSV Gewinner, im JSON winnerIndex und im Code winnerAgent. Der JSON-Export enthält kein `clockMode`, obwohl der Head-Hash nur bei logischer Uhr vergleichbar ist. Der Stresstest-Export nutzt standardmäßig die gleichverteilte Ground Truth (Proxy 0.409 bis 0.418), obwohl der aussagekräftigere Modus calm (Proxy 0.440 bis 0.905) verfügbar ist.

Warum das geändert werden soll

Gleiche Begriffe in allen Exporten erleichtern jede spätere Auswertung. Der Modus im Dateinamen und in einer Spalte verhindert, dass zwei Stresstest-Dateien mit verschiedener Ground Truth unbemerkt verglichen werden.

Änderung (Vorschlag, an eure Struktur anpassen)

```
// studies.ts: Exportschema upo-run-2 (altes Schema bleibt lesbar)
// JSON: clockMode ("utc" | "logical"), winnerIndex in allen Exporten gleich benennen
// CSV: Spalten Modus (uniform | calm) und Aktion_gebunden (true/false)
// Dateiname: upo-stress-50-calm.csv
// UI: Auswahl Ground Truth im Stresstest-Export, Standard weiterhin uniform
```

Abnahmekriterium

JSON und CSV nutzen dieselben Feldnamen. Die Stresstest-CSV trägt Modus und Aktion_gebunden. Ein Export im Schema upo-run-1 wird weiterhin vom Replay akzeptiert.

4. Arbeitsauftrag-Vorlage (Beispiel M15)

```
Aufgabe: Setze Block M15 aus dem Dokument "UPO v3.3.0 Nachtrag M15 bis M17" um
(Dateien: scenario.ts, adapter.ts, config.ts, selfcheck.ts).
Regeln:
1. Nur genannte Dateien aendern. Gewinner und Konfidenzen bitgleich zu C19 halten.
2. Eintrag C20 (kind: "correction") in assumptions.ts: Was, warum, Messwerte
   (90/90 Treffer, Marge min 0.128).
3. Tests T12 bis T14 zuerst gegen den alten Code laufen lassen (muessen fehlschlagen),
   danach gegen den neuen Code.
4. Die Schwelle ueber die gebundene Menge kalibrieren und im Export ausweisen.
5. Ausgabe: Diff-Zusammenfassung und die Messwerte aus dem Abnahmekriterium.
```

Ehrliche Grenzen: Die Messungen stammen aus Läufen des Kerns in Node. Die Szenario-Belegung der Aktionen ist ein Vorschlag, keine fachliche Wahrheit; und M16 ist ein ungemessener Vorschlag, dessen Abnahme das Ergebnis erst zeigt.

5. Sauber in GitHub eintragen

Ich gehe davon aus, dass du GitHub meinst. Grundlage ist die aktuell sichtbare Startseite des Repos (DarkStarMass/unified-process-ontology-v3.2.2): Ordner `data`, `operator/v3.3.0`, `src` und die README, 22 Commits, keine Releases oder Tags sichtbar, weder LICENSE noch CHANGELOG. Alles unten lässt sich auch im Browser oder in der GitHub-App erledigen (Dateien hochladen, Release anlegen). Die bestehenden Pfade bleiben unverändert, es kommt nur Ordnung dazu.

5.1 Befund und Vorschlag

Befund	Vorschlag
Repo-Name und About-Text sagen v3.2.2, der Inhalt ist 3.3.0-Master	About-Text aktualisieren; optional Repo in unified-process-ontology umbenennen (GitHub leitet die alte Adresse weiter)
README-Tabelle endet bei C14; C15 bis C19 stehen schon in assumptions.ts	Pro Korrektur eine Zeile in der README-Tabelle, Wortlaut aus assumptions.ts übernehmen
Keine Tags, keine Releases	Tag je abgenommenem Block (v3.3.0-C19, v3.3.0-C20, ...), pro Meilenstein ein Release mit PDF und JSON
Kein CHANGELOG	CHANGELOG.md mit einer Zeile je C-ID (Datum, Block, Messwert)
Analyse-PDFs und Messdateien liegen nicht im Repo	docs/analysen und docs/messungen, nummeriert und mit Datum im Namen
Herkunft von data/zustandsvektoren.csv ist nicht erklärt	data/README.md: stammt aus der Colab-Variante ohne Agent-0-Inversion, nicht aus src/unified_process_ontology_v3_2_2.py
Vertrauensmodell steht nicht in der README	SECURITY.md: Kette = Manipulationsnachweis nur mit extern gespeichertem Head-Hash; Replay = Determinismus-Test, kein Echtheitsbeweis
Keine LICENSE	Entscheidung bei dir; ohne Lizenz sind die Nutzungsrechte für andere nicht geklärt

5.2 Zielstruktur (bestehendes bleibt, Neues kommt dazu)

```
README.md                    Kurzfassung, Status, Tabelle C1..Cn, Links
CHANGELOG.md                 eine Zeile je Korrektur (C-ID, Block, Datum, Messwert)
SECURITY.md                  Meldeweg und Vertrauensmodell (Kette, Replay)
LICENSE                      Entscheidung offen
docs/analysen/               PDFs, nummeriert, mit Datum im Namen
docs/messungen/              JSON/CSV-Exporte mit Datum, Seed und Modus im Namen
docs/bloecke/                je Block eine kurze .md (M15.md ...): Befund, Warum, Abnahme
data/                        zustandsvektoren.csv plus data/README.md (Herkunft)
operator/v3.3.0/             Kern und Konsole (wie bisher)
src/                         eingefroren (v3.2.2, v3_2_6)
.github/ISSUE_TEMPLATE/block.md     Vorlage fuer einen Block
```

5.3 Ablauf je Block

Schritt	Was	Wo
1	Issue aus der Vorlage anlegen (ID, Befund, Abnahmekriterium)	GitHub: Issues
2	Branch anlegen, z. B. block/m15-aktionsprototypen	GitHub oder lokal
3	Tests zuerst gegen den alten Code laufen lassen (müssen fehlschlagen)	selfcheck.ts
4	Code ändern, Tests grün, in der App prüfen	operator/v3.3.0/src
5	Korrektur C-ID in assumptions.ts eintragen	assumptions.ts

Schritt	Was	Wo
6	README-Tabelle und CHANGELOG ergänzen	README.md, CHANGELOG.md
7	Messdateien ablegen (Datum, Seed, Modus im Namen)	docs/messungen/
8	Commit, in main zusammenführen, Tag setzen	GitHub: Releases > Draft a new release
9	Issue schließen, Status im Block-Dokument auf abgenommen setzen	Issues, docs/bloecke/

5.4 Namen und Texte

```
docs/analysen/01_2026-10-13_Tiefenanalyse_v3.2.2.pdf
docs/analysen/02_2026-10-14_Verbesserungen_v3.3.0.pdf
docs/analysen/03_2026-10-15_Nachtrag_M10-M14.pdf
docs/analysen/04_2026-10-17_Nachtrag_M15-M17.pdf
docs/messungen/2026-10-17_lauf_seed42_uniform.json
docs/messungen/2026-10-17_stress50_seed42_calm.csv

Commit:  C20: Aktionsprototypen aus Referenzszenarien (M15)
         test: T12-T14 fuer gebundene Aktionen (zuerst rot)
         docs: README-Tabelle C20, CHANGELOG, Messdateien 2026-10-17
Tag:     v3.3.0-C20     Branch: block/m15-aktionsprototypen
```

```
## 2026-10-17  C20  (M15)  Aktionsvektoren aus Referenzszenarien      <- Beispiel
- Messung: 90/90 Treffer, Marge min 0.128, Schwelle 0.0916
- Dateien: scenario.ts, adapter.ts, config.ts, selfcheck.ts
- Messdateien: docs/messungen/2026-10-17_lauf_seed42_calm.json
```

```
---
name: Block
about: Ein Verbesserungsblock (M-ID)
---
ID: M__    Korrektur: C__
Befund (gemessen):
Warum:
Dateien:
Abnahmekriterium (Zahlen):
Status: offen / in Arbeit / abgenommen
```

5.5 README-Ergänzung (Entwurf)

Zeilen für die Tabelle *Corrections against the PDF master*. Die Zuordnung folgt meinen Blöcken M10 bis M14; den genauen Wortlaut bitte aus `assumptions.ts` übernehmen:

```
| C15 | Ground Truth wahlweise aus Referenzszenario (calm): Distanz = Abweichung vom Ruhezustand. |
| C16 | Aktionsschwelle aus Nullverteilung (Quantil 0.99) statt fester 0.05. |
| C17 | toleranceCritical und toleranceHeadroom; degenerate genau dann, wenn tol >= tolCrit. |
| C18 | Kennzahl swarmDiversity: 1 minus mittlerer paarweiser Kosinus ohne Sonde. |
| C19 | JSON-Import gehaertet (Limit, Schemapruefung, nur bekannte Felder); headHash im Export. |
```

Vor jedem Push

Keine .env, keine Schlüssel, kein Plattform-Glue (Auth, Datenbank) im Commit. Mit `git status` und der Dateiliste des Commits prüfen. In den Repo-Einstellungen unter Security das Secret scanning aktivieren. In den Exporten keine Namen oder Pfade mit persönlichen Angaben lassen. Ich kann die Dateien nicht selbst ins Repo schieben, helfe aber gern bei README-, CHANGELOG- und SECURITY-Texten.